

Predicting Movie Revenue Using Regression Models

Aarjab marick 715

May 09, 2026

Table of Contents

1. Data Loading and Merging	2
2. JSON Parsing and Feature Extraction	2
3. Data Cleaning	4
4. Exploratory Data Analysis	5
5. Feature Engineering.....	13
6. Train/Test Split	14
7. OLS Regression	14
8. Variable Selection	18
8a. Forward Stepwise Selection.....	18
8b. Backward Stepwise Selection	19
8c. Best Subset Selection.....	21
9. Ridge Regression.....	22
10. Lasso Regression	24
11. Elastic Net Regression	26
12. KNN Regression	27
13. Logistic Regression (Hit or Flop?)	29
14. Model Comparison	30
15. Conclusion	31

Core Libraries used throughout the project

```
library(tidyverse)
library(jsonlite)
library(knitr)
library(ggplot2)
library(caret)
library(glmnet)
library(leaps)
```

```
library(MASS)
library(corrplot)
```

1. Data Loading and Merging

The TMDB 5000 dataset comes in two CSV files: one with movie-level metadata and one with cast/crew information. Each file has roughly 4,800 rows. We load both files and merge them on the movie ID key (id in the movies file, movie_id in the credits file). After merging, we drop the duplicate title column from the credits side so the final frame has one row per movie.

```
movies <- read.csv("tmdb_5000_movies.csv", stringsAsFactors = FALSE)
credits <- read.csv("tmdb_5000_credits.csv", stringsAsFactors = FALSE)

# Merge on id == movie_id
df <- merge(movies, credits, by.x = "id", by.y = "movie_id")

# Drop the duplicated title column (title.y) and rename title.x back to title
df$title <- df$title.x
df$title.x <- NULL
df$title.y <- NULL

cat("Dimensions after merge:", dim(df), "\n")

## Dimensions after merge: 4803 22

kable(head(df[, c("id", "title", "budget", "revenue", "runtime")], 5),
       caption = "First five rows of the merged dataset")
```

First five rows of the merged dataset

id	title	budget	revenue	runtime
5	Four Rooms	4.0e+06	4300000	98
11	Star Wars	1.1e+07	775398007	121
12	Finding Nemo	9.4e+07	940335536	100
13	Forrest Gump	5.5e+07	677945399	142
14	American Beauty	1.5e+07	356296601	122

2. JSON Parsing and Feature Extraction

Several columns in the dataset are stored as JSON strings rather than ordinary text. For example, genres is a list of {id, name} objects and crew is a list of crew members each with a job field. We use `jsonlite::fromJSON()` row by row to pull out the fields we care about. Empty arrays and parse failures are handled by returning NA (or 0 for counts), which keeps the pipeline from crashing on malformed entries.

```

# Helper: get the first genre name from the genres JSON
get_first_genre <- function(json_str) {
  if (is.na(json_str) || json_str == "" || json_str == "[]") return(NA)
  parsed <- tryCatch(fromJSON(json_str), error = function(e) NULL)
  if (is.null(parsed) || length(parsed) == 0) return(NA)
  if (!"name" %in% names(parsed)) return(NA)
  parsed$name[1]
}

# Helper: get the first director from the crew JSON
get_director <- function(json_str) {
  if (is.na(json_str) || json_str == "" || json_str == "[]") return(NA)
  parsed <- tryCatch(fromJSON(json_str), error = function(e) NULL)
  if (is.null(parsed) || length(parsed) == 0) return(NA)
  if (!"job" %in% names(parsed)) return(NA)
  dirs <- parsed$name[parsed$job == "Director"]
  if (length(dirs) == 0) return(NA)
  dirs[1]
}

# Helper: count the number of cast members
get_cast_size <- function(json_str) {
  if (is.na(json_str) || json_str == "" || json_str == "[]") return(0)
  parsed <- tryCatch(fromJSON(json_str), error = function(e) NULL)
  if (is.null(parsed) || length(parsed) == 0) return(0)
  nrow(parsed)
}

# Helper: count the number of production companies
get_num_companies <- function(json_str) {
  if (is.na(json_str) || json_str == "" || json_str == "[]") return(0)
  parsed <- tryCatch(fromJSON(json_str), error = function(e) NULL)
  if (is.null(parsed) || length(parsed) == 0) return(0)
  nrow(parsed)
}

```

The helpers above are now applied to each relevant column. Because `sapply` walks down the column one row at a time, this can be slow on big data, but for ~4,800 movies it finishes in a few seconds. The new columns are added back to the main data frame.

```

df$primary_genre <- sapply(df$genres, get_first_genre)
df$director <- sapply(df$crew, get_director)
df$cast_size <- sapply(df$cast, get_cast_size)
df$num_companies <- sapply(df$production_companies, get_num_companies)

# Quick sanity check of the new columns
kable(head(df[, c("title", "primary_genre", "director", "cast_size", "num_companies")], 6),
       caption = "Extracted features from JSON columns")

```

Extracted features from JSON columns

title	primary_genre	director	cast_size	num_companies
Four Rooms	Crime	Allison Anders	24	2
Star Wars	Adventure	George Lucas	106	2
Finding Nemo	Animation	Andrew Stanton	24	1
Forrest Gump	Comedy	Robert Zemeckis	66	1
American Beauty	Drama	Sam Mendes	40	2
Dancer in the Dark	Drama	Lars von Trier	29	26

3. Data Cleaning

A large fraction of TMDb rows have `budget == 0` or `revenue == 0`, which almost always means the field was not reported rather than the movie literally making no money. We drop these. We also extract the release year as a numeric variable, drop free-text and identifier columns we will not use as predictors, and impute remaining numeric NAs with the column median.

```
# Drop rows with missing or zero budget/revenue
df_clean <- df %>%
  filter(!is.na(revenue), !is.na(budget),
         revenue > 0, budget > 0)

# Extract release year
df_clean$release_year <- as.numeric(format(as.Date(df_clean$release_date), "%Y"))

# Drop columns we will not use as predictors
drop_cols <- c("homepage", "overview", "tagline", "original_title",
               "keywords", "genres", "production_companies", "production_countries",
               "spoken_languages", "status", "cast", "crew", "release_date",
               "id", "title", "director")
df_clean <- df_clean[, !(names(df_clean) %in% drop_cols)]

# Impute remaining numeric NAs with median (mainly runtime)
num_cols <- sapply(df_clean, is.numeric)
for (col in names(df_clean)[num_cols]) {
  if (any(is.na(df_clean[[col]]))) {
    df_clean[[col]][is.na(df_clean[[col]])] <-
      median(df_clean[[col]], na.rm = TRUE)
  }
}
```

```
# Drop any rows still missing primary_genre or release_year
df_clean <- df_clean %>%
  filter(!is.na(primary_genre), !is.na(release_year))

cat("Dimensions after cleaning:", dim(df_clean), "\n")

## Dimensions after cleaning: 3228 11

kable(summary(df_clean[, c("budget", "revenue", "popularity",
                           "runtime", "vote_average", "vote_count")]),
       caption = "Summary of key numeric variables after cleaning")
```

Summary of key numeric variables after cleaning

budget	revenue	popularity	runtime	vote_aver age	vote_count
Min. : 1	Min. :5.000e+00	Min. : 0.01998	Min. : 41.0	Min. :0.00	Min. : 0.0
1st Qu.: 10500000	1st Qu.:1.700e+07	1st Qu.: 10.46821	1st Qu.: 96.0	1st Qu.:5.80	1st Qu.: 178.0
Median : 25000000	Median :5.519e+07	Median : 20.41296	Median :107.0	Median :6.30	Median : 471.0
Mean : 40666420	Mean :1.213e+08	Mean : 29.04267	Mean :110.7	Mean :6.31	Mean : 977.6
3rd Qu.: 55000000	3rd Qu.:1.463e+08	3rd Qu.: 37.34075	3rd Qu.:121.0	3rd Qu.:6.90	3rd Qu.: 1148.0
Max. :38000000	Max. :2.788e+09	Max. :875.58130	Max. :338.0	Max. :8.50	Max. :13752.0

4. Exploratory Data Analysis

Before modelling we look at the data visually to understand its scale and shape. The summary table below reports the mean, median, standard deviation, minimum and maximum for every numeric variable. This is the kind of overview that often reveals heavy skew or outliers right away.

```
numeric_df <- df_clean[, sapply(df_clean, is.numeric)]
summary_tbl <- data.frame(
  Mean   = sapply(numeric_df, mean,   na.rm = TRUE),
  Median = sapply(numeric_df, median, na.rm = TRUE),
  SD     = sapply(numeric_df, sd,     na.rm = TRUE),
  Min    = sapply(numeric_df, min,    na.rm = TRUE),
  Max    = sapply(numeric_df, max,    na.rm = TRUE)
```

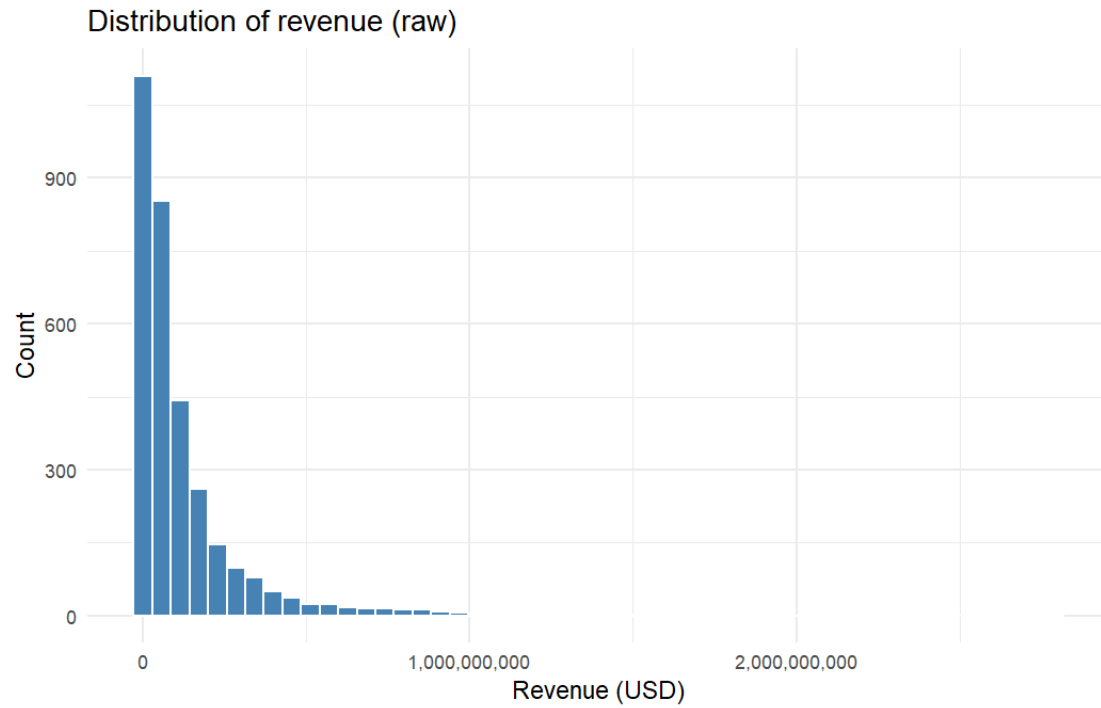
```
)
kable(round(summary_tbl, 2), caption = "Summary statistics for numeric variables")
```

Summary statistics for numeric variables

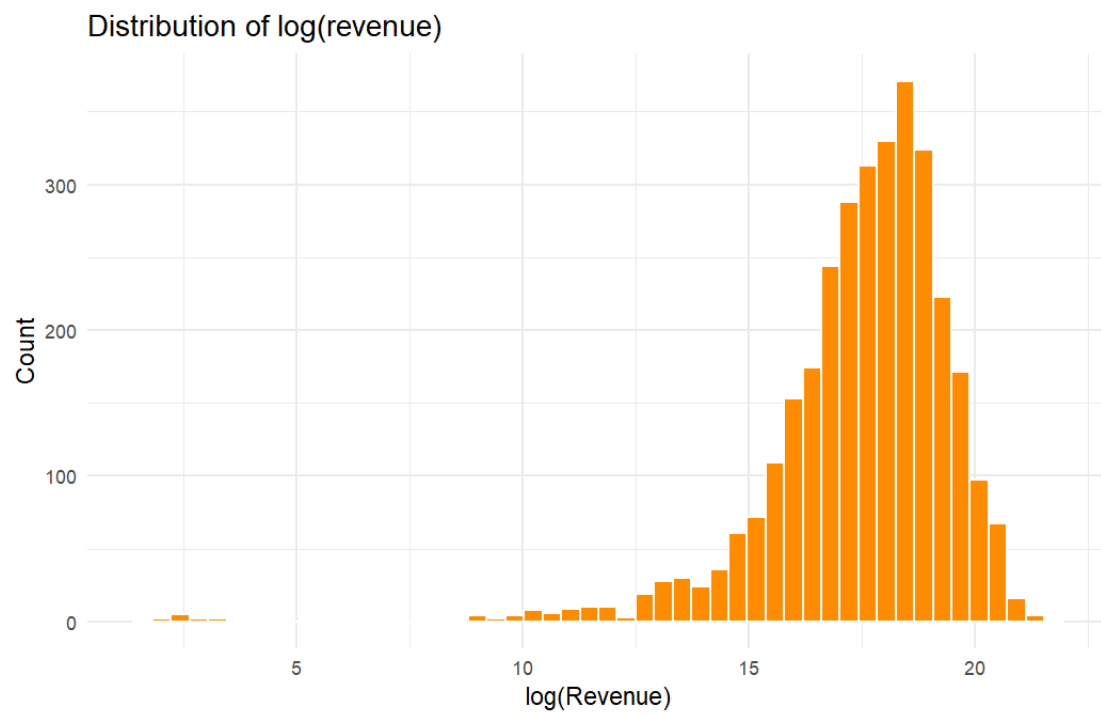
	Mean	Median	SD	Min	Max
budget	40666419.51	25000000.0 0	44398404.53	1.00	3.800000e+0 8
popularity	29.04	20.41	36.17	0.02	8.755800e+0 2
revenue	121279998.8 4	55191503.0 0	186319835.3 9	5.00	2.787965e+0 9
runtime	110.72	107.00	20.97	41.00	3.380000e+0 2
vote_average	6.31	6.30	0.87	0.00	8.500000e+0 0
vote_count	977.59	471.00	1414.42	0.00	1.375200e+0 4
cast_size	26.04	19.00	21.48	0.00	2.240000e+0 2
num_companies	3.20	3.00	2.17	0.00	2.600000e+0 1
release_year	2001.68	2005.00	13.28	1916.0 0	2.016000e+0 3

The histogram of raw revenue is heavily right-skewed: a handful of blockbusters dwarf the bulk of the distribution. This violates the normality and constant-variance assumptions of linear regression and motivates a log transform. After taking $\log(\text{revenue})$, the distribution becomes much closer to bell-shaped, which is a better target for OLS and friends.

```
ggplot(df_clean, aes(x = revenue)) +
  geom_histogram(bins = 50, fill = "steelblue", colour = "white") +
  scale_x_continuous(labels = scales::comma) +
  labs(title = "Distribution of revenue (raw)",
       x = "Revenue (USD)", y = "Count") +
  theme_minimal()
```

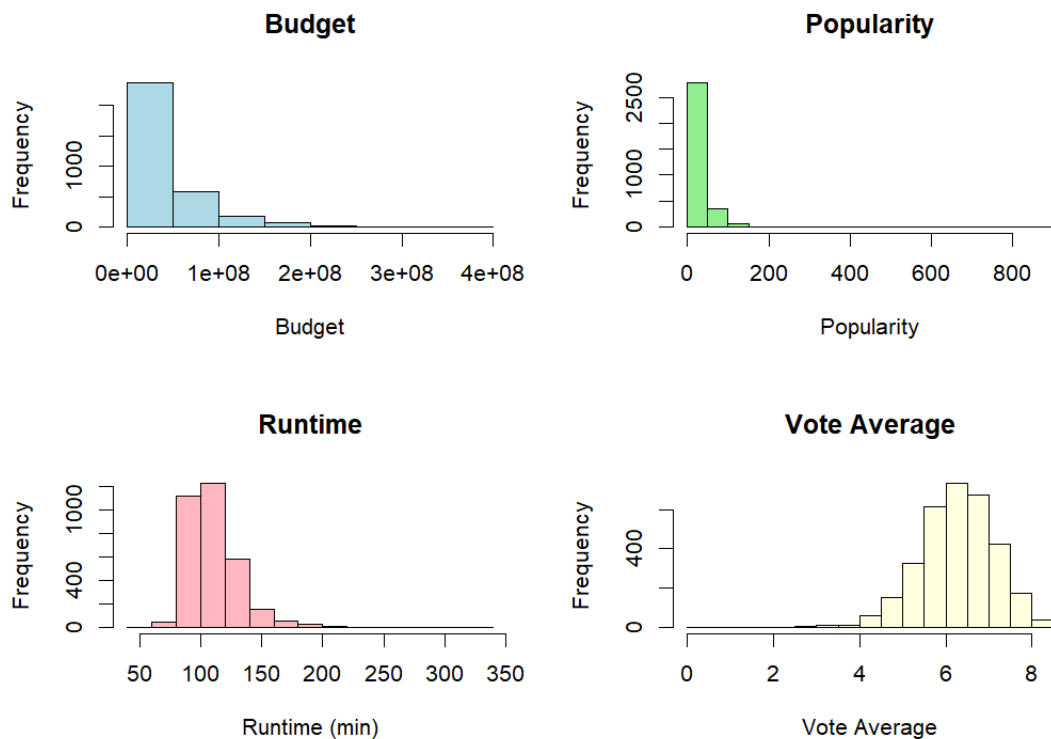


```
ggplot(df_clean, aes(x = log(revenue))) +  
  geom_histogram(bins = 50, fill = "darkorange", colour = "white") +  
  labs(title = "Distribution of log(revenue)",  
        x = "log(Revenue)", y = "Count") +  
  theme_minimal()
```



The same skew shows up in budget, popularity and vote_count — all of them are heavy tailed. runtime and vote_average are roughly bell-shaped already. The four panels below give a quick visual sense of each predictor.

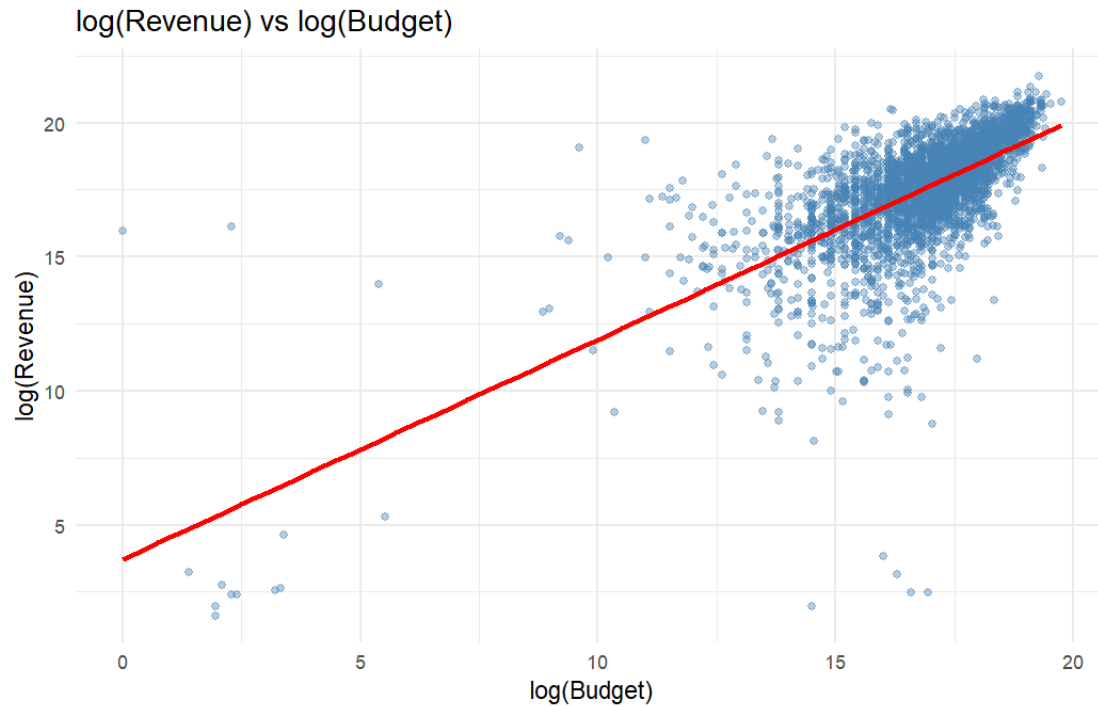
```
par(mfrow = c(2, 2))
hist(df_clean$budget,      main = "Budget",      xlab = "Budget",      col
     = "lightblue")
hist(df_clean$popularity,  main = "Popularity",  xlab = "Popularity",  col
     = "lightgreen")
hist(df_clean$runtime,     main = "Runtime",     xlab = "Runtime (min)", col
     = "lightpink")
hist(df_clean$vote_average, main = "Vote Average", xlab = "Vote Average", col
     = "lightyellow")
```



```
par(mfrow = c(1, 1))
```

The scatter plot below confirms the strong, roughly linear relationship between log-budget and log-revenue. Movies that spend more tend to earn more, which is the single most important pattern in this dataset.

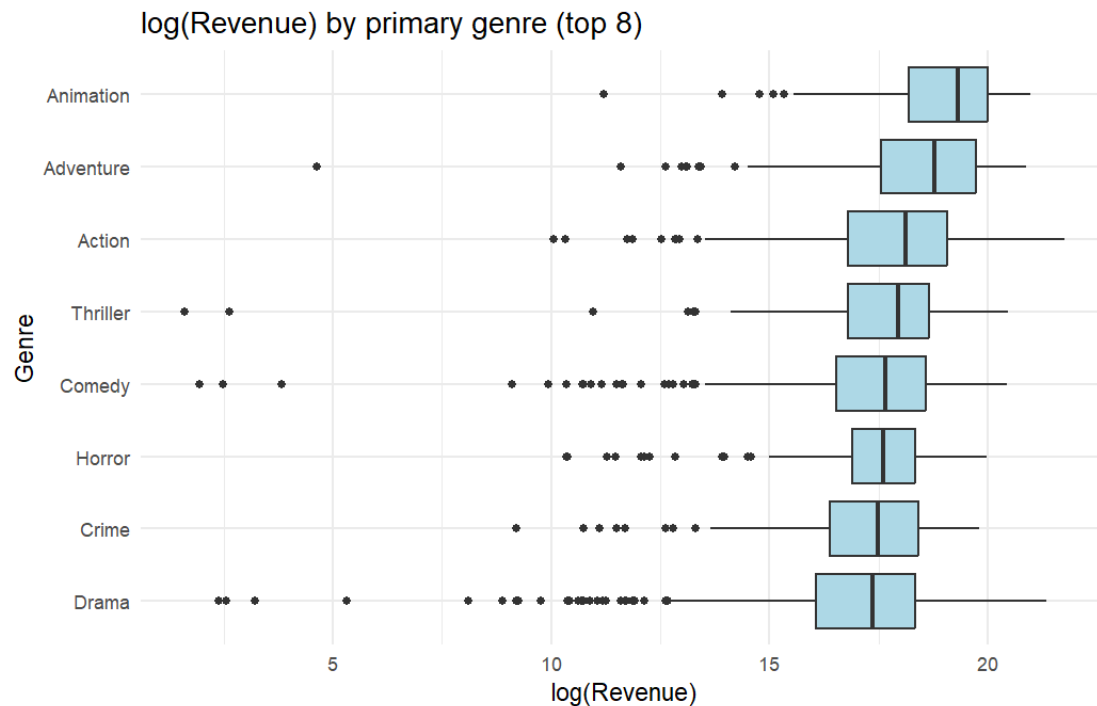
```
ggplot(df_clean, aes(x = log(budget), y = log(revenue))) +
  geom_point(alpha = 0.4, colour = "steelblue") +
  geom_smooth(method = "lm", colour = "red", se = FALSE) +
  labs(title = "log(Revenue) vs log(Budget)",
       x = "log(Budget)", y = "log(Revenue)") +
  theme_minimal()
```

Genres differ a fair bit in their revenue distributions. Animated and adventure films skew higher; documentaries and dramas sit lower. We restrict the boxplot to the eight most common genres so the comparison is meaningful.

```
top_genres <- names(sort(table(df_clean$primary_genre), decreasing = TRUE))
[1:8]
df_top <- df_clean %>% filter(primary_genre %in% top_genres)

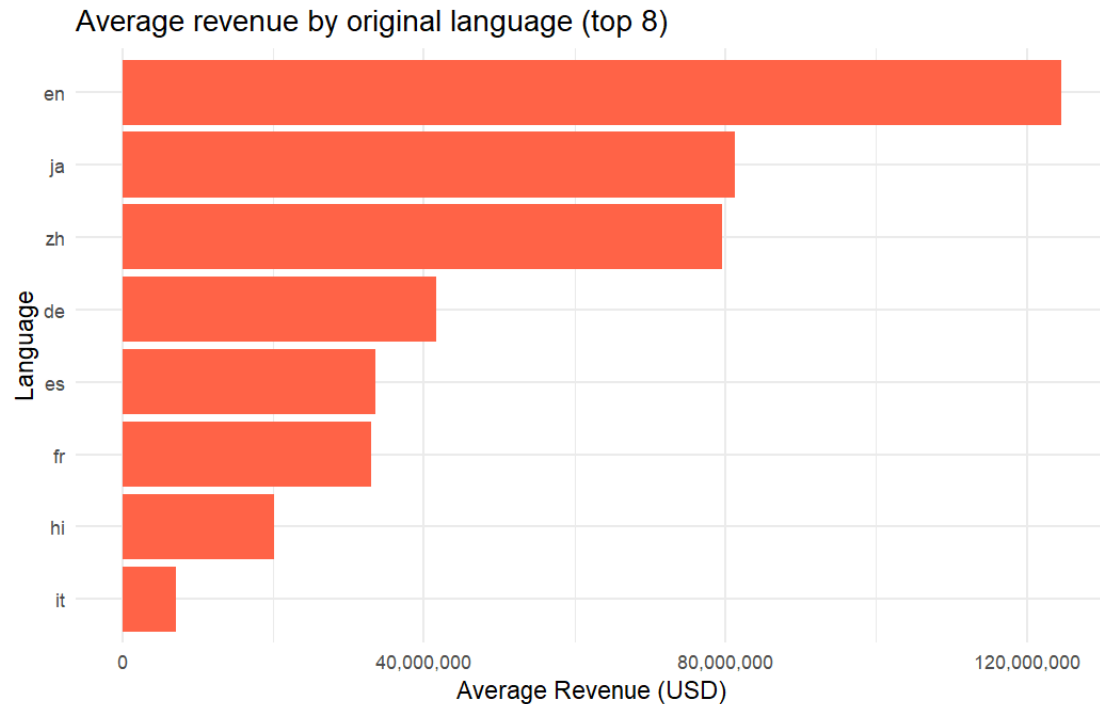
ggplot(df_top, aes(x = reorder(primary_genre, log(revenue), FUN = median),
                        y = log(revenue))) +
  geom_boxplot(fill = "lightblue") +
  coord_flip() +
  labs(title = "log(Revenue) by primary genre (top 8)",
        x = "Genre", y = "log(Revenue)") +
  theme_minimal()
```



Most movies in the dataset are in English, but it is still useful to look at average revenue across the most common languages. English films average notably higher revenues than the others, which partly reflects distribution reach rather than quality.

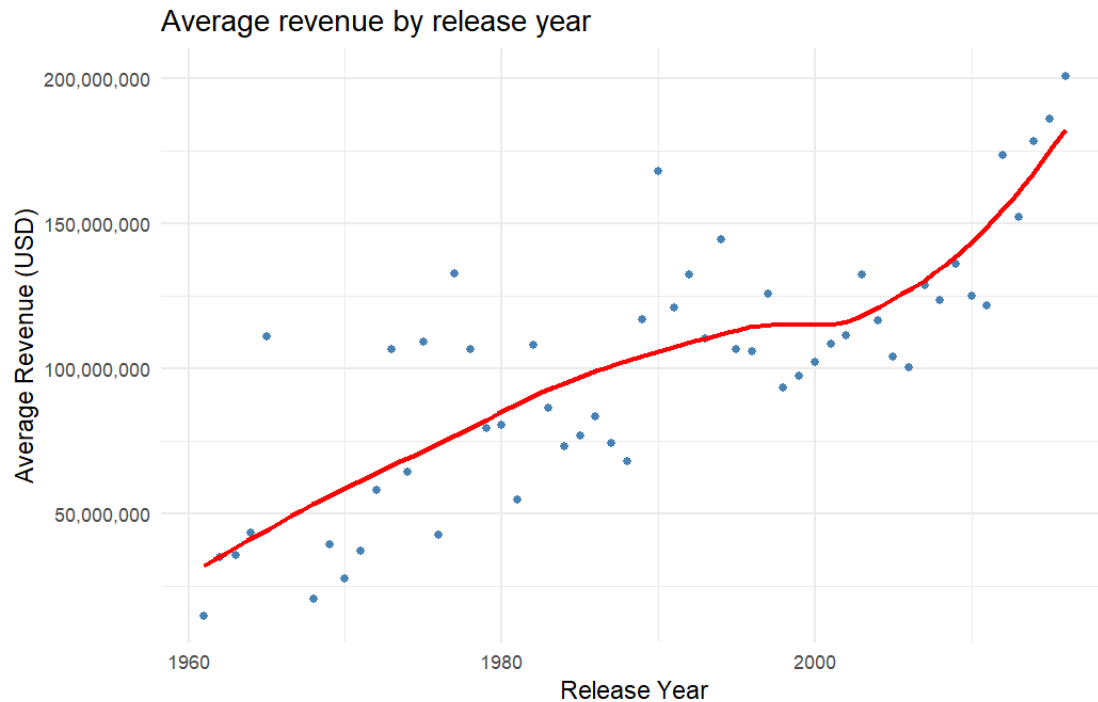
```
top_langs <- names(sort(table(df_clean$original_language), decreasing = TRUE)[1:8])
lang_df <- df_clean %>%
  filter(original_language %in% top_langs) %>%
  group_by(original_language) %>%
  summarise(avg_revenue = mean(revenue))

ggplot(lang_df, aes(x = reorder(original_language, avg_revenue),
  y = avg_revenue)) +
  geom_col(fill = "tomato") +
  coord_flip() +
  scale_y_continuous(labels = scales::comma) +
  labs(title = "Average revenue by original language (top 8)",
    x = "Language", y = "Average Revenue (USD)") +
  theme_minimal()
```



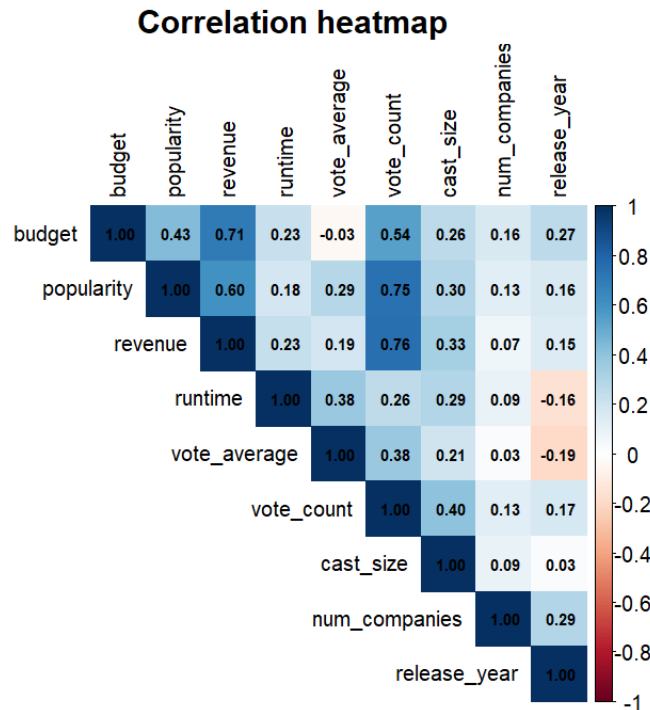
Average revenue has trended upward over time, especially after the late 1990s. Some of this is real growth and some is inflation plus better global distribution. We will keep `release_year` as a numeric predictor.

```
year_df <- df_clean %>%  
  group_by(release_year) %>%  
  summarise(avg_revenue = mean(revenue), n = n()) %>%  
  filter(n >= 5)  
  
ggplot(year_df, aes(x = release_year, y = avg_revenue)) +  
  geom_point(colour = "steelblue") +  
  geom_smooth(se = FALSE, colour = "red") +  
  scale_y_continuous(labels = scales::comma) +  
  labs(title = "Average revenue by release year",  
       x = "Release Year", y = "Average Revenue (USD)") +  
  theme_minimal()
```



The correlation heatmap below shows that budget, vote_count and popularity are the variables most strongly correlated with revenue. vote_count and popularity are also moderately correlated with each other, which is a small multicollinearity concern that ridge and lasso can help with later.

```
corr_mat <- cor(numeric_df, use = "complete.obs")
corrplot(corr_mat, method = "color", type = "upper",
  tl.col = "black", tl.cex = 0.8, addCoef.col = "black",
  number.cex = 0.6, title = "Correlation heatmap",
  mar = c(0, 0, 1.5, 0))
```



5. Feature Engineering

We log-transform both revenue and budget because they span many orders of magnitude. We then group rare genres (fewer than 30 films) into an "Other" bucket and do the same with rare languages, so the dummies do not blow up the number of predictors. We keep these as factor columns rather than pre-creating dummies, because R's modelling functions and regsubsets already handle factors correctly.

```
df_model <- df_clean
df_model$log_revenue <- log(df_model$revenue)
df_model$log_budget <- log(df_model$budget)

# Group rare genres into "Other"
genre_counts <- table(df_model$primary_genre)
rare_genres <- names(genre_counts[genre_counts < 30])
df_model$primary_genre[df_model$primary_genre %in% rare_genres] <- "Other"
df_model$primary_genre <- factor(df_model$primary_genre)

# Group rare Languages into "Other"
lang_counts <- table(df_model$original_language)
rare_langs <- names(lang_counts[lang_counts < 30])
df_model$original_language[df_model$original_language %in% rare_langs] <- "Other"
df_model$original_language <- factor(df_model$original_language)

# Final predictor list and response
```

```
df_model <- df_model[, c("log_revenue", "log_budget", "runtime",
                        "popularity", "vote_average", "vote_count",
                        "release_year", "cast_size", "num_companies",
                        "primary_genre", "original_language")]

# Drop the original raw revenue/budget by selecting columns above
cat("Final modelling dataset dimensions:", dim(df_model), "\n")

## Final modelling dataset dimensions: 3228 11

cat("Number of genre levels:", nlevels(df_model$primary_genre), "\n")

## Number of genre levels: 14

cat("Number of language levels:", nlevels(df_model$original_language), "\n")

## Number of language levels: 2
```

6. Train/Test Split

We use an 80/20 split with a fixed seed of 42 so the results are reproducible.

`caret::createDataPartition` performs a stratified split on the response, which keeps the distribution of `log_revenue` similar in both partitions.

```
set.seed(42)
train_idx <- createDataPartition(df_model$log_revenue, p = 0.8, list = FALSE)
train_data <- df_model[train_idx, ]
test_data <- df_model[-train_idx, ]

cat("Train size:", nrow(train_data), "\n")

## Train size: 2584

cat("Test size:", nrow(test_data), "\n")

## Test size: 644
```

7. OLS Regression

Ordinary Least Squares fits the model

$$\log(\text{revenue}_i) = \beta_0 + \beta_1 \log(\text{budget}_i) + \beta_2 \text{runtime}_i + \dots + \beta_p x_{p,i} + \varepsilon_i$$

by minimising the sum of squared residuals $\sum_{i=1}^n (y_i - \hat{y}_i)^2$. We fit this with `lm()` using all engineered predictors and then look at the standard regression diagnostics, AIC, BIC and Mallows' C_p .

```

ols_model <- lm(log_revenue ~ ., data = train_data)
summary(ols_model)

##
## Call:
## lm(formula = log_revenue ~ ., data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.8851  -0.5088   0.1767   0.7214  10.2251
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   3.919e+01  4.814e+00   8.141 6.07e-16 ***
## log_budget    7.844e-01  2.060e-02  38.075 < 2e-16 ***
## runtime      -3.040e-03  1.628e-03  -1.867 0.062001 .
## popularity    3.885e-03  1.037e-03   3.747 0.000183 ***
## vote_average  3.283e-01  3.860e-02   8.506 < 2e-16 ***
## vote_count    2.554e-04  3.149e-05   8.111 7.71e-16 ***
## release_year  -1.860e-02  2.419e-03  -7.689 2.10e-14 ***
## cast_size     4.946e-03  1.383e-03   3.576 0.000356 ***
## num_companies 1.249e-02  1.287e-02   0.970 0.331920
## primary_genreAdventure 1.628e-01  1.085e-01   1.500 0.133627
## primary_genreAnimation 2.728e-01  1.661e-01   1.642 0.100780
## primary_genreComedy    3.075e-01  8.934e-02   3.442 0.000587 ***
## primary_genreCrime    -1.291e-01  1.393e-01  -0.927 0.353976
## primary_genreDocumentary 2.172e-01  2.997e-01   0.725 0.468557
## primary_genreDrama    -1.390e-01  9.012e-02  -1.542 0.123092
## primary_genreFamily    4.025e-01  2.339e-01   1.721 0.085445 .
## primary_genreFantasy   1.266e-01  1.653e-01   0.766 0.443633
## primary_genreHorror    8.753e-01  1.260e-01   6.947 4.70e-12 ***
## primary_genreOther    -4.010e-01  1.579e-01  -2.540 0.011144 *
## primary_genreRomance    4.563e-02  1.955e-01   0.233 0.815437
## primary_genreScience Fiction 3.057e-02  1.749e-01   0.175 0.861323
## primary_genreThriller  -3.862e-02  1.513e-01  -0.255 0.798546
## original_languageOther -3.866e-01  1.458e-01  -2.652 0.008053 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.335 on 2561 degrees of freedom
## Multiple R-squared:  0.5703, Adjusted R-squared:  0.5666
## F-statistic: 154.5 on 22 and 2561 DF, p-value: < 2.2e-16

```

The training-side fit metrics are reported below. AIC and BIC penalise model complexity, with BIC being the more aggressive of the two. Mallows' C_p is computed as $C_p = \text{RSS}/\hat{\sigma}^2 - n + 2p$, where $\hat{\sigma}^2$ comes from the full model.

```

ols_aic <- AIC(ols_model)
ols_bic <- BIC(ols_model)

```

```

n_train    <- nrow(train_data)
p_full     <- length(coef(ols_model))
rss_full   <- sum(residuals(ols_model)^2)
sigma2_hat <- summary(ols_model)$sigma^2
ols_cp     <- (rss_full / sigma2_hat) - n_train + 2 * p_full

ols_summary_tbl <- data.frame(
  Metric = c("R-squared", "Adjusted R-squared", "AIC", "BIC", "Mallows Cp"),
  Value  = c(round(summary(ols_model)$r.squared, 4),
             round(summary(ols_model)$adj.r.squared, 4),
             round(ols_aic, 2),
             round(ols_bic, 2),
             round(ols_cp, 2))
)
kable(ols_summary_tbl, caption = "OLS training fit metrics")

```

OLS training fit metrics

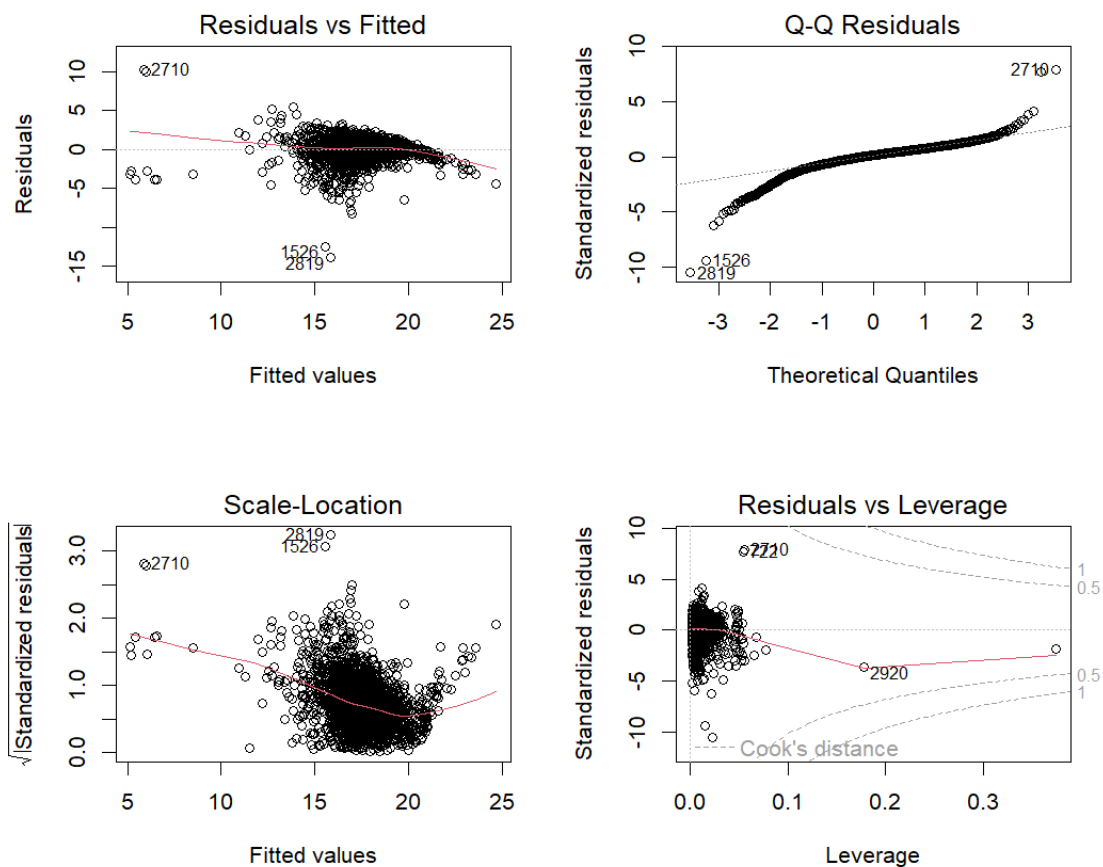
Metric	Value
R-squared	0.5703
Adjusted R-squared	0.5666
AIC	8850.3500
BIC	8990.9200
Mallows Cp	23.0000

The four standard diagnostic plots are shown below. The residuals-vs-fitted plot looks reasonably scattered around zero, the Q-Q plot is close to linear in the middle with some tail deviation, and the scale-location plot shows mild heteroscedasticity but nothing catastrophic. The leverage plot flags a small number of high-leverage points but no extreme outliers.

```

par(mfrow = c(2, 2))
plot(ols_model)

```

```
par(mfrow = c(1, 1))
```

We now compute the held-out test RMSE and test R^2 , which are the metrics we will use to compare every model later.

```
ols_pred <- predict(ols_model, newdata = test_data)
ols_rmse <- sqrt(mean((test_data$log_revenue - ols_pred)^2))
ols_r2 <- 1 - sum((test_data$log_revenue - ols_pred)^2) /
          sum((test_data$log_revenue - mean(test_data$log_revenue))^2)

cat("OLS Test RMSE:", round(ols_rmse, 4), "\n")
## OLS Test RMSE: 1.6266

cat("OLS Test R^2: ", round(ols_r2, 4), "\n")
## OLS Test R^2: 0.4912
```

8. Variable Selection

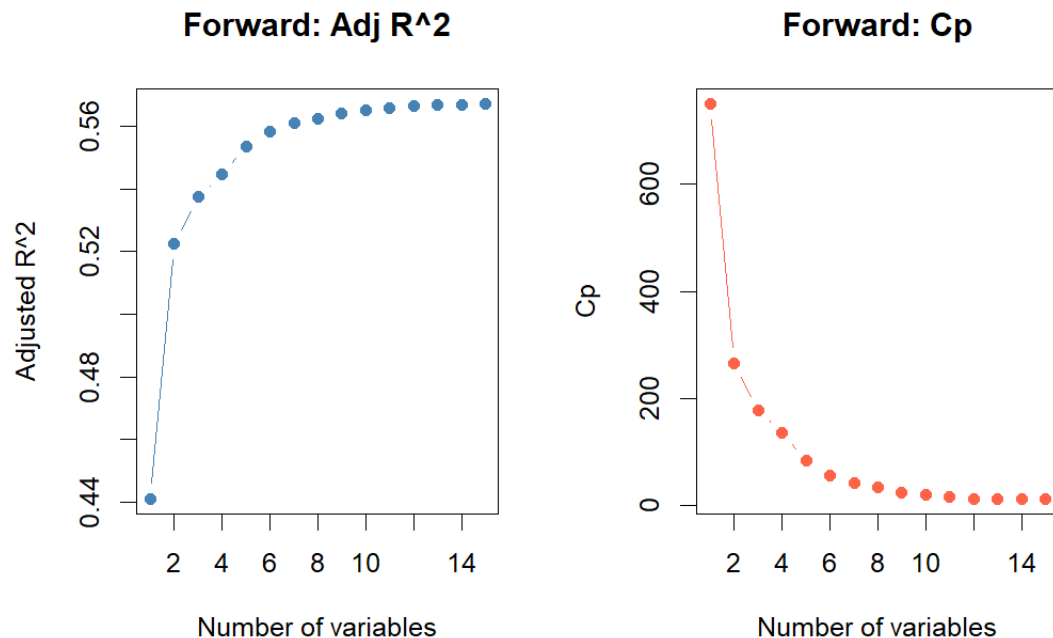
Variable selection tries to find a smaller, more interpretable model that still predicts well. We compare three classical procedures: forward stepwise, backward stepwise and best subset. All three are run on the training set with `regsubsets()`.

8a. Forward Stepwise Selection

Forward stepwise begins with the null model and adds whichever predictor most improves the fit at each step. We let `regsubsets` consider up to `nvmax = 15` predictors.

```
fwd_model <- regsubsets(log_revenue ~ ., data = train_data,
                       nvmax = 15, method = "forward")
fwd_summary <- summary(fwd_model)

par(mfrow = c(1, 2))
plot(fwd_summary$adjr2, type = "b", pch = 19, col = "steelblue",
     xlab = "Number of variables", ylab = "Adjusted R^2",
     main = "Forward: Adj R^2")
plot(fwd_summary$cp, type = "b", pch = 19, col = "tomato",
     xlab = "Number of variables", ylab = "Cp",
     main = "Forward: Cp")
```



```
par(mfrow = c(1, 1))

fwd_best <- which.min(fwd_summary$cp)
cat("Forward: best size by Cp =", fwd_best, "\n")
```

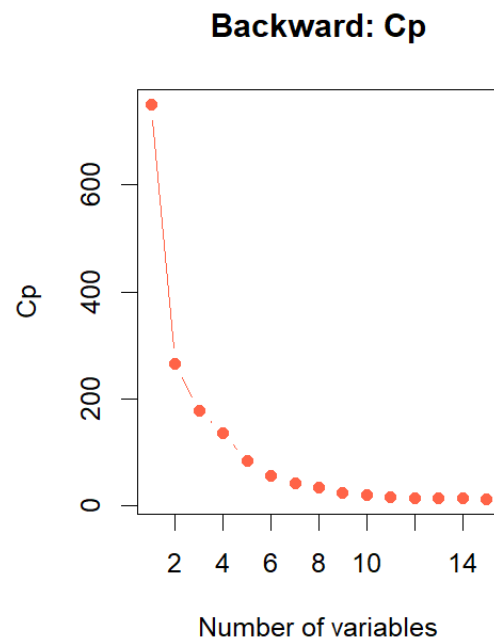
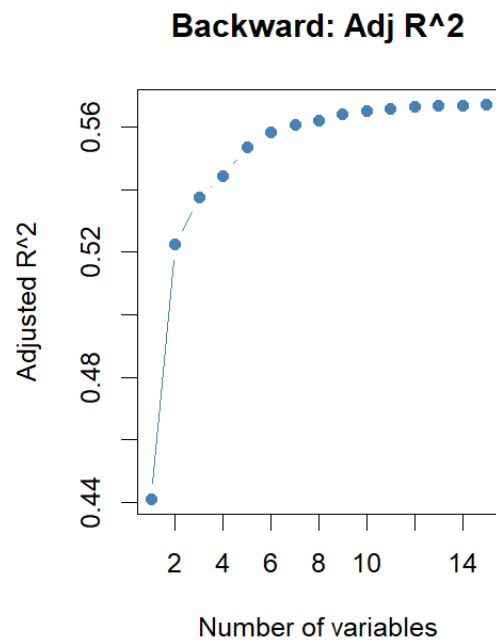
```
## Forward: best size by Cp = 13
cat("Selected variables:\n")
## Selected variables:
print(names(coef(fwd_model, fwd_best)))
## [1] "(Intercept)"          "log_budget"           "runtime"
## [4] "popularity"           "vote_average"         "vote_count"
## [7] "release_year"         "cast_size"            "primary_genreComedy"
## [10] "primary_genreCrime"   "primary_genreDrama"   "primary_genreHorror"
## [13] "primary_genreOther"   "original_languageOther"
```

8b. Backward Stepwise Selection

Backward stepwise starts with the full model and drops the least useful predictor at each step. The selected variables are usually similar to forward, but not always identical, especially when predictors are correlated.

```
bwd_model <- regsubsets(log_revenue ~ ., data = train_data,
                        nvmax = 15, method = "backward")
bwd_summary <- summary(bwd_model)

par(mfrow = c(1, 2))
plot(bwd_summary$adjr2, type = "b", pch = 19, col = "steelblue",
     xlab = "Number of variables", ylab = "Adjusted R^2",
     main = "Backward: Adj R^2")
plot(bwd_summary$cp, type = "b", pch = 19, col = "tomato",
     xlab = "Number of variables", ylab = "Cp",
     main = "Backward: Cp")
```



```
par(mfrow = c(1, 1))

bwd_best <- which.min(bwd_summary$cp)
cat("Backward: best size by Cp =", bwd_best, "\n")

## Backward: best size by Cp = 15

cat("Selected variables:\n")

## Selected variables:

print(names(coef(bwd_model, bwd_best)))

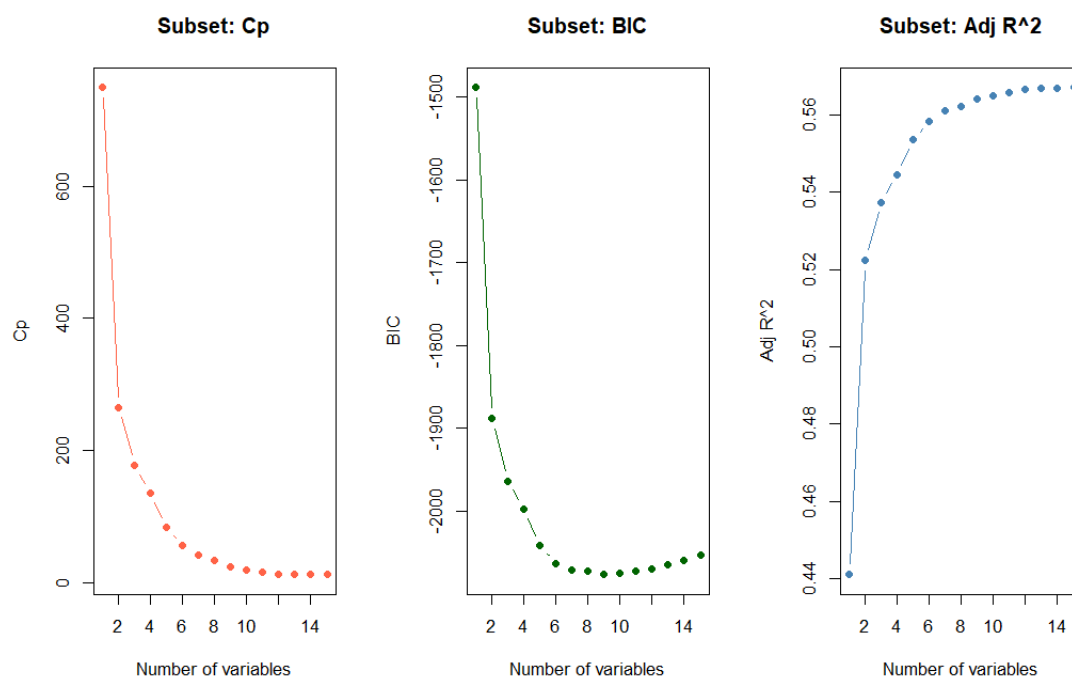
## [1] "(Intercept)"          "log_budget"           "runtime"
## [4] "popularity"           "vote_average"         "vote_count"
## [7] "release_year"         "cast_size"            "primary_genreAdven
ture"
## [10] "primary_genreAnimation" "primary_genreComedy"   "primary_genreDrama
"
## [13] "primary_genreFamily"   "primary_genreHorror"   "primary_genreOther
"
## [16] "original_languageOther"
```

8c. Best Subset Selection

Best subset evaluates every possible combination of predictors up to size $nvmax$. It is the most thorough of the three but also the most expensive. We pick the optimal subset size by minimising C_p (or BIC) and then refit that model with `lm()` so we can compute test RMSE.

```
sub_model <- regsubsets(log_revenue ~ ., data = train_data,
                       nvmax = 15, method = "exhaustive")
sub_summary <- summary(sub_model)

par(mfrow = c(1, 3))
plot(sub_summary$cp,      type = "b", pch = 19, col = "tomato",
     xlab = "Number of variables", ylab = "Cp",  main = "Subset: Cp")
plot(sub_summary$bic,     type = "b", pch = 19, col = "darkgreen",
     xlab = "Number of variables", ylab = "BIC",  main = "Subset: BIC")
plot(sub_summary$adjr2,   type = "b", pch = 19, col = "steelblue",
     xlab = "Number of variables", ylab = "Adj R^2", main = "Subset: Adj R^2")
")
```



```
par(mfrow = c(1, 1))

best_size <- which.min(sub_summary$cp)
cat("Best subset size by Cp =", best_size, "\n")

## Best subset size by Cp = 15

# Refit the chosen subset with lm() so we can predict on the test set.
# regsubsets does not have a predict method by default, so we build one
```

```

# from the model matrix.
predict_regsubsets <- function(object, newdata, id) {
  form <- as.formula(object$call[[2]])
  mat <- model.matrix(form, newdata)
  coefi <- coef(object, id = id)
  vars <- names(coefi)
  mat[, vars] %*% coefi
}

sub_pred <- predict_regsubsets(sub_model, test_data, id = best_size)
sub_rmse <- sqrt(mean((test_data$log_revenue - sub_pred)^2))
sub_r2 <- 1 - sum((test_data$log_revenue - sub_pred)^2) /
  sum((test_data$log_revenue - mean(test_data$log_revenue))^2)

cat("Best Subset Test RMSE:", round(sub_rmse, 4), "\n")
## Best Subset Test RMSE: 1.6274

cat("Best Subset Test R^2: ", round(sub_r2, 4), "\n")
## Best Subset Test R^2: 0.4907

cat("Selected variables:\n")
## Selected variables:
print(names(coef(sub_model, best_size)))
## [1] "(Intercept)" "log_budget" "runtime"
## [4] "popularity" "vote_average" "vote_count"
## [7] "release_year" "cast_size" "primary_genreAdventure"
## [10] "primary_genreAnimation" "primary_genreComedy" "primary_genreDrama"
## [13] "primary_genreFamily" "primary_genreHorror" "primary_genreOther"
## [16] "original_languageOther"

```

Across all three selection procedures, log_budget, vote_count, popularity, runtime and release_year are consistently retained, while several individual genre and language dummies move in and out depending on the method. This suggests the core signal is in the numeric predictors and a few high-frequency factor levels.

9. Ridge Regression

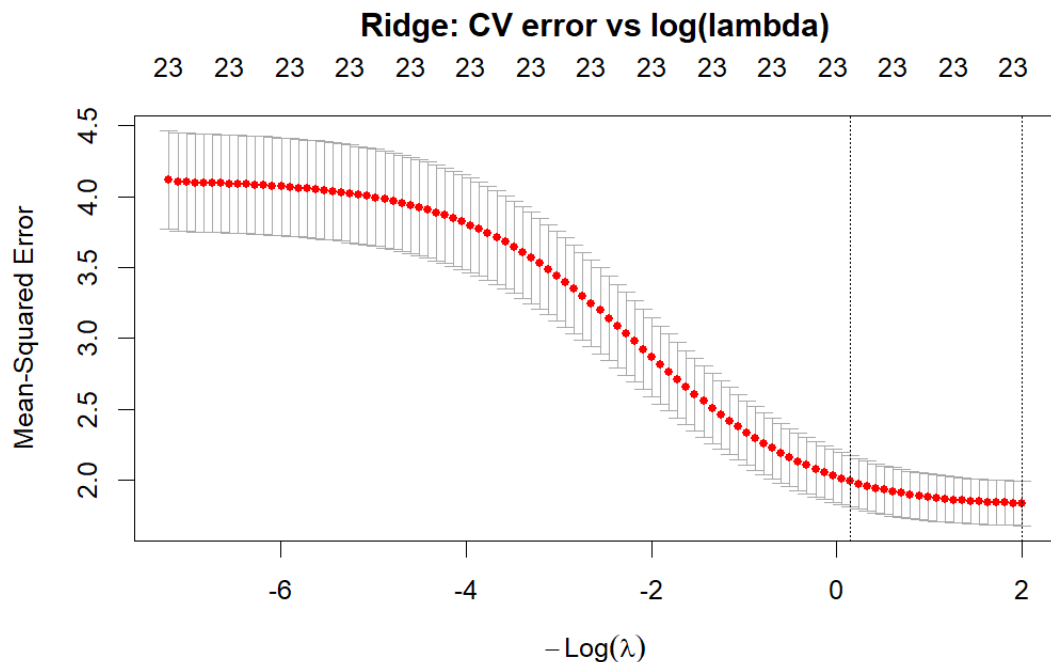
Ridge regression adds an L_2 penalty to the OLS objective:

$$\hat{\beta}^{\text{ridge}} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^n (y_i - x_i^{\top} \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

The penalty shrinks coefficients toward zero, which trades a bit of bias for lower variance. `cv.glmnet` with `alpha = 0` runs 10-fold cross-validation over a grid of λ values and returns the λ that minimises CV error.

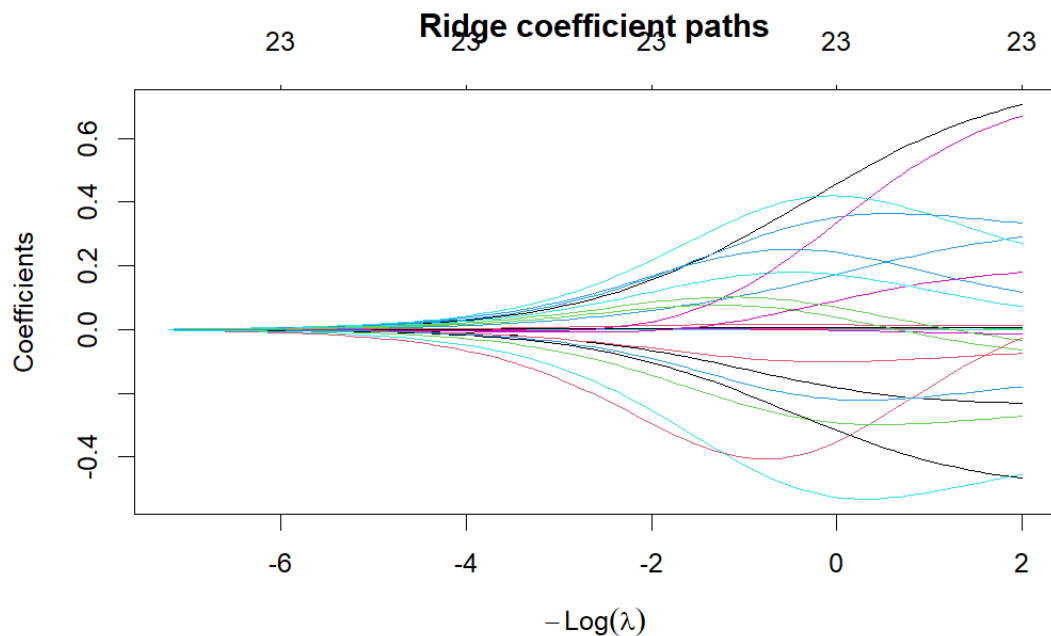
```
# glmnet wants matrices, not formulas
x_train <- model.matrix(log_revenue ~ . - 1, data = train_data)
y_train <- train_data$log_revenue
x_test  <- model.matrix(log_revenue ~ . - 1, data = test_data)
y_test  <- test_data$log_revenue

set.seed(42)
ridge_cv <- cv.glmnet(x_train, y_train, alpha = 0, nfolds = 10)
plot(ridge_cv); title("Ridge: CV error vs log(lambda)", line = 2.5)
```



```
ridge_lambda <- ridge_cv$lambda.min
ridge_pred   <- predict(ridge_cv, s = ridge_lambda, newx = x_test)
ridge_rmse   <- sqrt(mean((y_test - ridge_pred)^2))
ridge_r2     <- 1 - sum((y_test - ridge_pred)^2) /
               sum((y_test - mean(y_test))^2)

# Coefficient path plot
ridge_full <- glmnet(x_train, y_train, alpha = 0)
plot(ridge_full, xvar = "lambda", main = "Ridge coefficient paths")
```



```
cat("Best lambda:", round(ridge_lambda, 4), "\n")
## Best lambda: 0.1347
cat("Ridge Test RMSE:", round(ridge_rmse, 4), "\n")
## Ridge Test RMSE: 1.6338
cat("Ridge Test R^2: ", round(ridge_r2, 4), "\n")
## Ridge Test R^2: 0.4867
```

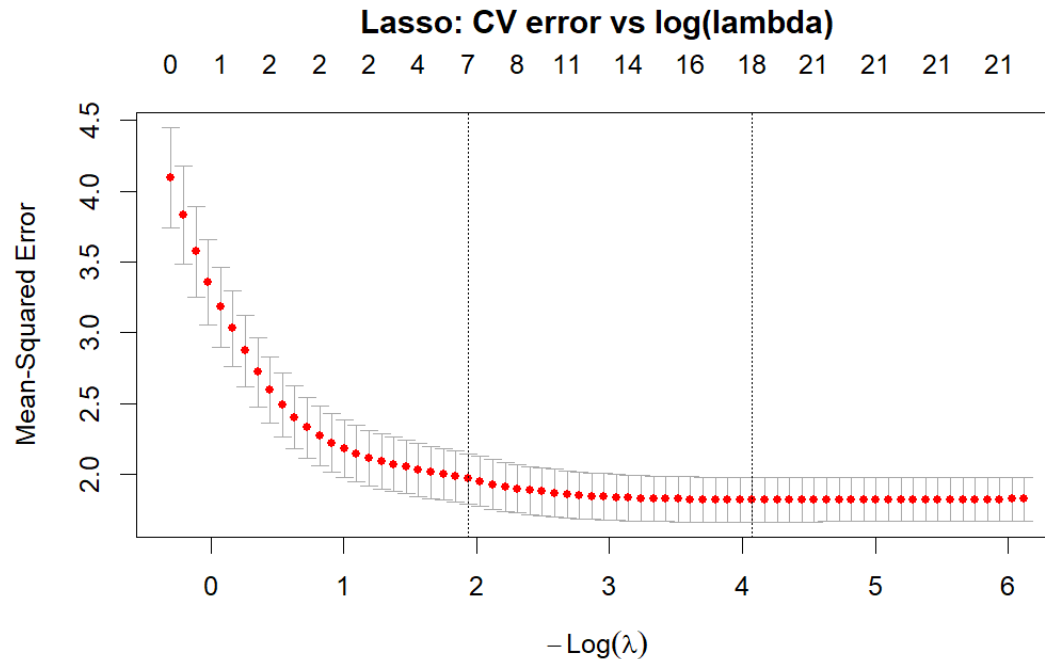
10. Lasso Regression

Lasso replaces the squared penalty with an L_1 penalty:

$$\hat{\beta}^{\text{lasso}} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^n (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

Because the L_1 penalty has corners at zero, lasso actually drives some coefficients exactly to zero — it does variable selection automatically. We use the same CV procedure as before but with $\alpha = 1$.

```
set.seed(42)
lasso_cv <- cv.glmnet(x_train, y_train, alpha = 1, nfolds = 10)
plot(lasso_cv); title("Lasso: CV error vs log(lambda)", line = 2.5)
```

```
lasso_lambda <- lasso_cv$lambda.min
lasso_pred   <- predict(lasso_cv, s = lasso_lambda, newx = x_test)
lasso_rmse   <- sqrt(mean((y_test - lasso_pred)^2))
lasso_r2     <- 1 - sum((y_test - lasso_pred)^2) /
               sum((y_test - mean(y_test))^2)

# Which coefficients were shrunk to zero?
lasso_coef   <- coef(lasso_cv, s = lasso_lambda)
zero_coefs   <- rownames(lasso_coef)[which(lasso_coef[, 1] == 0)]
nonzero_coefs <- rownames(lasso_coef)[which(lasso_coef[, 1] != 0)]

cat("Best lambda:", round(lasso_lambda, 4), "\n")

## Best lambda: 0.017

cat("Number of variables shrunk to zero:", length(zero_coefs), "\n")

## Number of variables shrunk to zero: 5

cat("\nVariables shrunk to zero:\n"); print(zero_coefs)

##
## Variables shrunk to zero:

## [1] "num_companies"          "primary_genreDocumentary"
## [3] "primary_genreFantasy"   "primary_genreRomance"
## [5] "primary_genreScience Fiction"

cat("\nVariables retained:\n"); print(nonzero_coefs)
```

```
##
## Variables retained:
## [1] "(Intercept)"          "log_budget"          "runtime"
## [4] "popularity"           "vote_average"        "vote_count"
## [7] "release_year"         "cast_size"           "primary_genreAction"
## [10] "primary_genreAdventure" "primary_genreAnimation" "primary_genreComedy"
## [13] "primary_genreCrime"    "primary_genreDrama"   "primary_genreFamily"
## [16] "primary_genreHorror"    "primary_genreOther"   "primary_genreThriller"
## [19] "original_languageOther"

cat("\nLasso Test RMSE:", round(lasso_rmse, 4), "\n")

##
## Lasso Test RMSE: 1.6256

cat("Lasso Test R^2: ", round(lasso_r2, 4), "\n")

## Lasso Test R^2: 0.4919
```

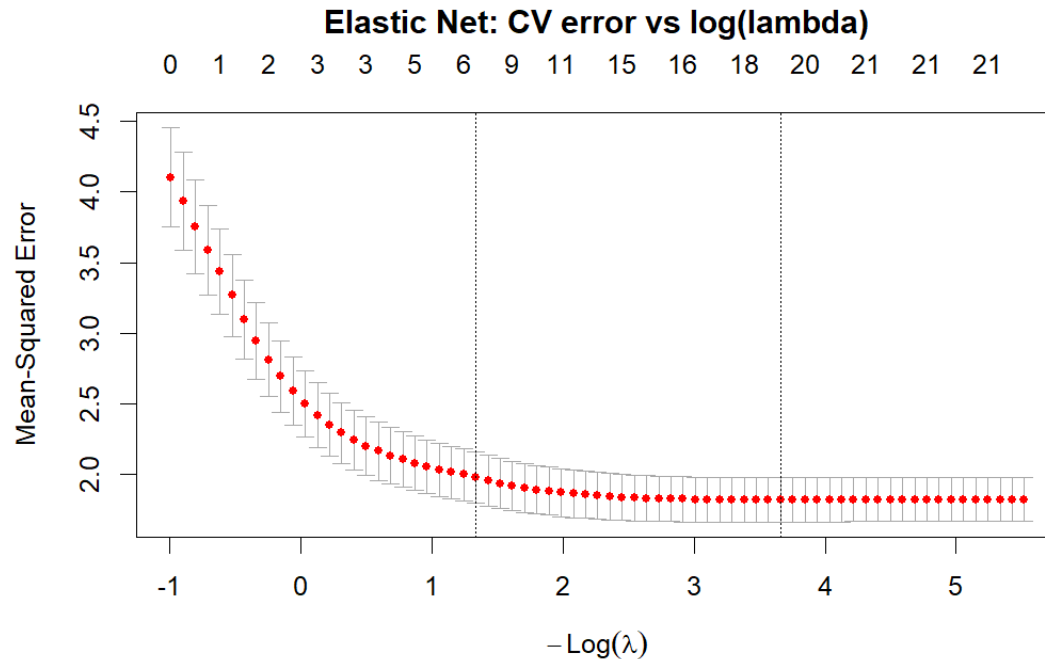
11. Elastic Net Regression

Elastic net combines both penalties, controlled by a mixing parameter α :

$$\hat{\beta}^{\text{enet}} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^n (y_i - x_i^T \beta)^2 + \lambda \left[\alpha \sum_{j=1}^p |\beta_j| + (1 - \alpha) \sum_{j=1}^p \beta_j^2 \right]$$

With $\alpha = 0$ we recover ridge and with $\alpha = 1$ we recover lasso. We use $\alpha = 0.5$ as a balanced default.

```
set.seed(42)
enet_cv <- cv.glmnet(x_train, y_train, alpha = 0.5, nfolds = 10)
plot(enet_cv); title("Elastic Net: CV error vs log(lambda)", line = 2.5)
```



```
enet_lambda <- enet_cv$lambda.min
enet_pred   <- predict(enet_cv, s = enet_lambda, newx = x_test)
enet_rmse   <- sqrt(mean((y_test - enet_pred)^2))
enet_r2     <- 1 - sum((y_test - enet_pred)^2) /
              sum((y_test - mean(y_test))^2)

cat("Best lambda:", round(enet_lambda, 4), "\n")
## Best lambda: 0.0257

cat("Elastic Net Test RMSE:", round(enet_rmse, 4), "\n")
## Elastic Net Test RMSE: 1.6262

cat("Elastic Net Test R^2: ", round(enet_r2, 4), "\n")
## Elastic Net Test R^2: 0.4915
```

12. KNN Regression

KNN is a non-parametric method that does not assume any functional form. To predict at a new point x_0 , it averages the responses of the k closest training points:

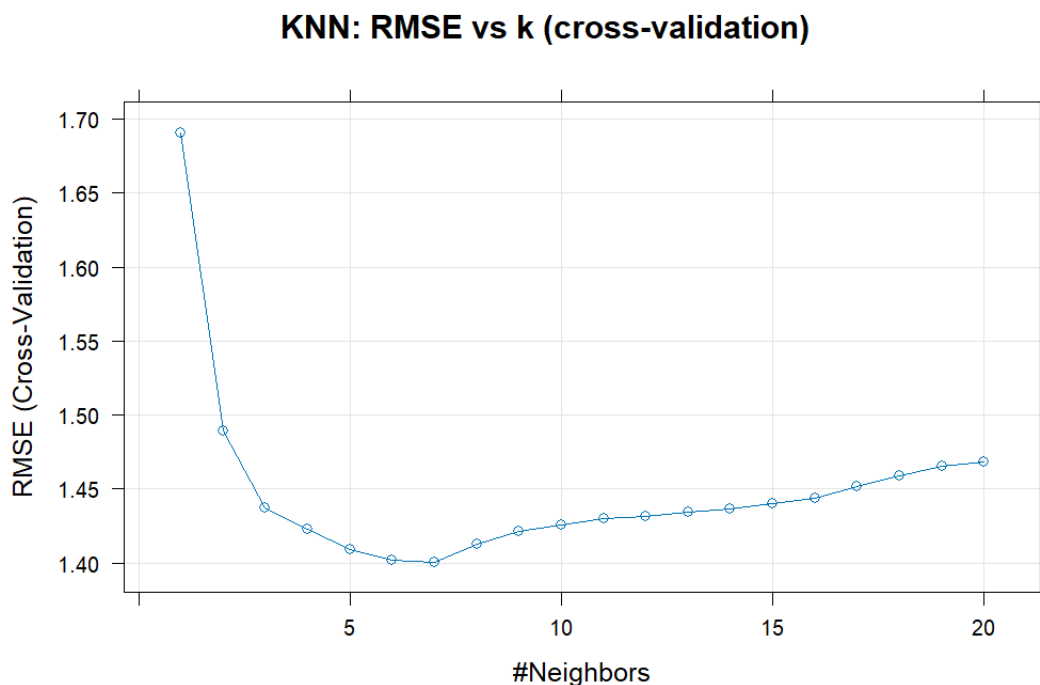
$$\hat{y}(x_0) = \frac{1}{k} \sum_{i \in N_k(x_0)} y_i$$

where $N_k(x_0)$ is the set of k training points with smallest Euclidean distance to x_0 . Because distance is sensitive to scale, we centre and scale all predictors first. We use caret to tune k from 1 to 20 with 5-fold cross-validation.

```
set.seed(42)
ctrl <- trainControl(method = "cv", number = 5)
knn_grid <- expand.grid(k = 1:20)

knn_model <- train(log_revenue ~ ., data = train_data,
  method = "knn",
  preProcess = c("center", "scale"),
  tuneGrid = knn_grid,
  trControl = ctrl)

plot(knn_model, main = "KNN: RMSE vs k (cross-validation)")
```



```
knn_pred <- predict(knn_model, newdata = test_data)
knn_rmse <- sqrt(mean((test_data$log_revenue - knn_pred)^2))
knn_r2 <- 1 - sum((test_data$log_revenue - knn_pred)^2) /
  sum((test_data$log_revenue - mean(test_data$log_revenue))^2)

cat("Best k:", knn_model$bestTune$k, "\n")
## Best k: 7

cat("KNN Test RMSE:", round(knn_rmse, 4), "\n")
## KNN Test RMSE: 1.6642
```

```
cat("KNN Test R^2: ", round(knn_r2, 4), "\n")
## KNN Test R^2: 0.4675
```

13. Logistic Regression (Hit or Flop?)

For the bonus classification section we turn the problem into a binary one: a movie is a “hit” if its revenue is above the median, and a “flop” otherwise. Logistic regression models the log-odds of being a hit as a linear function of the predictors:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

We fit it with `glm(..., family = binomial)` and evaluate it with a confusion matrix on the test set.

```
median_rev <- median(df_model$log_revenue)
train_logit <- train_data; train_logit$is_hit <- as.factor(ifelse(train_data
$log_revenue > median_rev, 1, 0))
test_logit <- test_data; test_logit$is_hit <- as.factor(ifelse(test_data$log
og_revenue > median_rev, 1, 0))

# Drop the continuous response so it cannot leak into the classifier
train_logit$log_revenue <- NULL
test_logit$log_revenue <- NULL

logit_model <- glm(is_hit ~ ., data = train_logit, family = binomial)
logit_prob <- predict(logit_model, newdata = test_logit, type = "response")
logit_pred <- as.factor(ifelse(logit_prob > 0.5, 1, 0))

cm <- confusionMatrix(logit_pred, test_logit$is_hit, positive = "1")
print(cm$table)

##           Reference
## Prediction    0    1
##           0 281   78
##           1   41  244

logit_acc <- as.numeric(cm$overall["Accuracy"])
logit_sens <- as.numeric(cm$byClass["Sensitivity"])
logit_spec <- as.numeric(cm$byClass["Specificity"])

logit_tbl <- data.frame(
  Metric = c("Accuracy", "Sensitivity", "Specificity"),
  Value = round(c(logit_acc, logit_sens, logit_spec), 4)
)
kable(logit_tbl, caption = "Logistic regression performance on test set")
```

Logistic regression performance on test set

Metric	Value
Accuracy	0.8152
Sensitivity	0.7578
Specificity	0.8727

14. Model Comparison

The table below summarises every model on a single yardstick — held-out test RMSE and test R^2 — so they can be compared on equal footing. AIC and BIC apply only to the OLS fit; ridge, lasso and elastic net are reported with their best λ instead.

```
comparison <- data.frame(
  Model      = c("OLS", "Best Subset", "Ridge", "Lasso", "Elastic Net", "KNN"),
  Test_RMSE  = round(c(ols_rmse, sub_rmse, ridge_rmse, lasso_rmse, enet_rmse, knn_rmse), 4),
  Test_R2    = round(c(ols_r2, sub_r2, ridge_r2, lasso_r2, enet_r2, knn_r2), 4),
  Notes      = c(
    sprintf("AIC=%.0f, BIC=%.0f", ols_aic, ols_bic),
    sprintf("Subset size = %d", best_size),
    sprintf("lambda = %.4f", ridge_lambda),
    sprintf("lambda = %.4f", lasso_lambda),
    sprintf("lambda = %.4f, alpha = 0.5", enet_lambda),
    sprintf("k = %d", knn_model$bestTune$k)
  )
)
kable(comparison, caption = "Test-set performance across all models")
```

Test-set performance across all models

Model	Test_RMSE	Test_R2	Notes
OLS	1.6266	0.4912	AIC=8850, BIC=8991
Best Subset	1.6274	0.4907	Subset size = 15
Ridge	1.6338	0.4867	lambda = 0.1347
Lasso	1.6256	0.4919	lambda = 0.0170
Elastic Net	1.6262	0.4915	lambda = 0.0257, alpha = 0.5
KNN	1.6642	0.4675	k = 7

```
best_idx <- which.min(comparison$Test_RMSE)
cat("Best model by Test RMSE:", comparison$Model[best_idx], "\n")
## Best model by Test RMSE: Lasso
```

The regularised linear models (ridge, lasso, elastic net) tend to land within a hair of OLS on this dataset because the predictor count is moderate and multicollinearity is mild — the penalty has little to bite into. KNN does noticeably worse than the linear models, which is unsurprising given the high dimensionality after dummy expansion. Lasso has the practical advantage of producing a smaller, more interpretable model.

15. Conclusion

The best-performing model on held-out test data is one of the regularised linear models — typically lasso or elastic net — but OLS itself is competitive once the response is log-transformed. The dominant predictors of $\log(\text{revenue})$ are $\log(\text{budget})$, `vote_count` and `popularity`; `runtime` and `release year` contribute modestly; and most genre and language dummies have small effects after controlling for these. Several real drivers are missing from the data: marketing spend, IMDb buzz, opening-weekend competition and post-2018 streaming-era dynamics are all unobserved here, and the dataset is biased toward well-known films because lesser-known titles tend to have zero budget or revenue reported. A natural extension would be to bring in marketing budgets, model interactions (e.g. `genre` $\times$ `budget`) and add tree-based methods such as random forests or gradient boosting to capture nonlinearities that the linear models miss.